

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



SOFTWARE-QUALITÄT UND SICHERHEITSFRAGEN

Themen in dieser Vorlesung



SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 1)

Einführung und Grundbegriffe

Software durchdringt die vernetzte Welt

Softwarefehler und ihre Folgen

Softwarefehler: Radiosender in Seattle schrottet Infotainment von Mazda

In Seattle hat ein Radiosender fehlerhafte Dateien versendet und Infotainment-Systeme von Mazda unbenutzbar gemacht. Der Sender lässt sich nicht mehr wechseln.

11.02.2022 13:56 Uhr

Von Martin Holland

Ein Radiosender in der US-Metropole Seattle hat einen Weg gefunden, Hörern und Hörerinnen dauerhaft die Möglichkeit zu nehmen, den Sender zu wechseln. Die "HD Radios" in einer unbekannten Zahl von Fahrzeugen der Marke Mazda weigern sich seit Ende Januar, einen anderen Sender abzuspielen als den Nachrichtensender KUOW-FM.

Abgesehen davon kann das Infotainment überhaupt nichts mehr, unter anderem das Navigationssystem, die Anzeige der Uhrzeit und die Rückspiegelkamera funktionieren nicht mehr. Öffentlich gemacht hat das die lokale Zeitung Seattle Times, nachdem Betroffene erst im Internet herausgefunden hatten, dass es sich

Software durchdringt die vernetzte Welt

Softwarefehler und ihre Folgen

Und wie war das noch mit der Boeing 737 MAX? ...

Softwarefehler: Radiosender in Seattle schrottet Infotainment

In Seattle hat ein Radiosender fehlerhafte Systeme von Mazda unbenutzbar gemacht. Die Fahrer müssen den Radiosender wechseln.

11.02.2022 13:56 Uhr

Von Martin Holland

Ein Radiosender in der US-Metropole Seattle hat die Fahrerinnen und Hörerinnen dauerhaft die Möglichkeit verloren, das Infotainment-System zu nutzen. Die "HD Radios" in einer unbekannten Anzahl von Fahrzeugen weigern sich seit Ende Januar, einen automatisierten Nachrichtensender KUOW-FM zu empfangen.

Abgesehen davon kann das Infotainment-System das Navigationssystem, die Anzeige der Uhrzeit und die Rücksichtnahme-Funktion nicht mehr steuern. Öffentlich gemacht hat das die lokale Zeitung Times, nachdem Betroffene erst im Internet herausgefunden hatten.

Y2K22-Bug legt Stromanschluss lahm – Kärntner ohne Warmwasser

Zum Jahreswechsel ist in 1.600 Kärntner Haushalten der Nachtstrom ausgefallen. Grund ist ein Softwarefehler in Schaltuhren.

04.01.2022 01:59 Uhr

Von Daniel AJ Sokolov

Am 1. Jänner, 0:00 Uhr, ist in rund 1.600 Haushalten im österreichischen Bundesland Kärnten der Nachtstrom ausgefallen. Schuld ist ein Softwarefehler in Schaltuhren, die etwa zehn Jahre alt sind. Damit haben diese Haushalte kein Warmwasser mehr und in manchen Fällen auch keine Heizung. Sie müssen auf den Hausbesuch eines Monteurs warten.

Das sind nur zwei typische von zahlreichen Berichten heutzutage.

Software durchdringt die vernetzte Welt

Softwareschwachstellen und ihre Folgen

Ransomware-Erpressung mehr als verdoppelt: 2021 über 600 Millionen Angriffe

Hacker verdreifachten die Zahl der Ransomware-Angriffe in den letzten drei Jahren, wie ein aktueller Report zeigt. Hauptziel der meisten Attacken sind die USA.

24.02.2022 11:49 Uhr | iX Magazin

Von Philipp Steevens

Die Zahl der Ransomware-Angriffe ist im Jahr 2021 im Vergleich zum Vorjahr um 105 Prozent gestiegen. AtlasVPN berichtet in seinem Blog, dass Kriminelle im letzten Jahr beinahe 20-mal die Sekunde versuchten, Systeme mit Ransomware zu verseuchen. Die Zahlen beruhen auf dem aktuellen 2022 SonicWall Cyber Threat Report. Glückt ein solcher Versuch, können Unternehmen und Anwender nicht mehr auf ihre Daten zugreifen, da die Hacker das System verschlüsseln. Die Erpresser fordern hohe Geldsummen, um die Daten wieder freizugeben oder kritische Firmengeheimnisse nicht zu veröffentlichen. Insgesamt wurden 2021 über 623 Millionen Angriffe mit Ransomware registriert.

Software durchdringt die vernetzte Welt

Softwareschwachstellen und ihre Folgen

Ransomware-Erpressung mehr als verdoppelt: 2021 üb

Hacker verdreifachten die Zahl der Jahren, wie ein aktueller Report ze USA.

24.02.2022 11:49 Uhr | iX Magazin

Von Philipp Steevens

Die Zahl der Ransomware-Angriffe 105 Prozent gestiegen. AtlasVPN be letzten Jahr beinahe 20-mal die Se zu verseuchen. Die Zahlen beruher Threat Report. Glückt ein solcher V nicht mehr auf ihre Daten zugreifen Erpresser fordern hohe Geldsumm kritische Firmengeheimnisse nicht über 623 Millionen Angriffe mit Ransomware registriert.

Cyberattacken auf Ziele in der Ukraine

Sicherheitsforscher von Eset warnen vor einer datenlöschenden Malware auf hunderten Rechnern. Währenddessen laufen DDoS-Angriffe auf ukrainische Webseiten.

24.02.2022 11:00 Uhr | Security

Von Dirk Knop

Den Angriff auf die Ukraine durch Russland begleiten Cyberangriffe auf ukrainische Computer und Webseiten. Sicherheitsforscher von Eset haben beobachtet, wie die Malware, die sie HermeticWiper getauft haben, seit dem gestrigen späten Nachmittag Daten von Rechnern gelöscht und die Maschinen anschließend neu gestartet hat. Zeitstempel der Dateien weisen auf eine mehrmonatige Vorbereitung.

Software durchdringt die vernetzte Welt

Fehler: Entstehung, Erkennung – und Kosten!

Normalisierte Kosten

Source	Phase Requirements Issue Found			
	Requirements	Design	Code	Test
[Boehm, 1981]	1	5	10	50
[Hoffman, 2001]	1	3	5	37
[Cigital, 2003]	1	3	7	51
[Rothman, 2000]		5	33	75
[Rothman, 2000] Case B			10	40
[Rothman, 2000] Case C			10	40
[Rothman, 2002]	1	20	45	250
[Pavlina, 2003]	1	10	100	1000
[McGibbon, 2003]		5		50
Mean	1	7.3	25.6	177
Median	1	5	10	50.5

Stecklein et. al.: *Error Cost Escalation Through the Project Life Cycle*. Nasa Technical Report, 2004.

Fehler

Was ist das?

- **Fehlverhalten:** Ein Fehlverhalten oder Ausfall (failure) zeigt sich dynamisch bei der Benutzung eines Produkts. Beim dynamischen Test einer Software erkennt man keine Fehler, sondern Fehlverhalten bzw. Ausfälle. Diese sind Wirkungen von Fehlern im Programm. Bei Hardware sind z. B. Ausfälle durch Verschleiß möglich. Synonym: Fehlfunktion. Ähnlich: Fehlzustand, ein durch einen Fehler hervorgerufener falscher Zustand während der Programmausführung
- **Fehler:** Ein Fehler oder Defekt (fault, defect) ist bei Software die statisch im Programmcode vorhandene Ursache eines Fehlverhaltens oder Ausfalls. Bei Hardware sind ebenfalls Konstruktionsfehler möglich (z. B. eine zu schwache Dimensionierung der Kühlung eines Leistungshalbleiters).

Software durchdringt die vernetzte Welt

Fehler und Sicherheit

Fehler sind oft auch Sicherheitsprobleme!

- Fehler können Schwachstellen in einer Software verursachen
- Über Schwachstellen kann man Programme und damit Computersysteme in vom Programmierer eigentlich unerwünschte Zustände versetzen
- Bis hin zur vollständigen Übernahme von Computern durch Fremde
- Und *ein* Fehler kann reichen...

Software durchdringt die vernetzte Welt

Mit wieviel Fehlern müssen wir rechnen?

Die Fehlerdichte, die Anzahl der Softwarefehler pro 1000 Zeilen Code, ist bei qualitativ hochwertiger Software heute im Schnitt 0,3. Da gängige Betriebssysteme ca. 10. Mio. Zeilen Code haben, sind hier im Schnitt 3000 Software-Fehler zu finden.

(Pohlmann, Norbert (2011) Bugs, die Nahrung für Malware – Von guter, schlechter und böser Software. IT-Sicherh Manage Prax 4:32–34 zitiert aus Pohlmann, Norbert (2019): Cyber-Sicherheit. Wiesbaden: Springer Fachmedien Wiesbaden.)

Software und Fehler, sichere Funktion, Hacks

Wir müssen reden!

- Moderne Gesellschaften sind von Software abhängig und durchdrungen, Computersysteme international eng verflochten
- Damit entsteht ein hoher Wert – und folglich Risiken, die bei der Erstellung von Software angemessen berücksichtigt werden müssen
- Wir haben folglich ein **großes Problem**, wenn...
 - ...bei der Erstellung von Software diesen Risiken nicht angemessen Rechnung getragen wird
 - ...Software nicht sicher ist
 - ...Software nicht verlässlich ist
 - ...allgemeiner: Software nicht „gut genug“ ist
- Dazu: Fehlerkosten sind ein echtes Problem für die Hersteller

Und was heißt nun: „gut genug“ und wie erreichen wir das?

Software-Qualität

Von lat. *qualitas* „Beschaffenheit, Eigenschaft“

Conformance to established software requirements; the capability of a software product to satisfy stated and implied needs when used under specified conditions

ISO 25010:2011 nach Laporte/April 2018

- IEEE 730-2014 ergänzt: *The degree to which a software product meets established requirements; however, quality depends upon the degree to which established requirements accurately represent stakeholder needs, wants, and expectations.*
 - Bezug auf Anforderungen, Bedürfnisse, festgelegte Nutzungsbedingungen
 - Aber, selbst mit Definition: Software-Qualität bleibt abstrakt!
 - Wie messen wir, wie vergleichen wir Qualität?

Software-Qualität

Qualitätsmodelle für Software

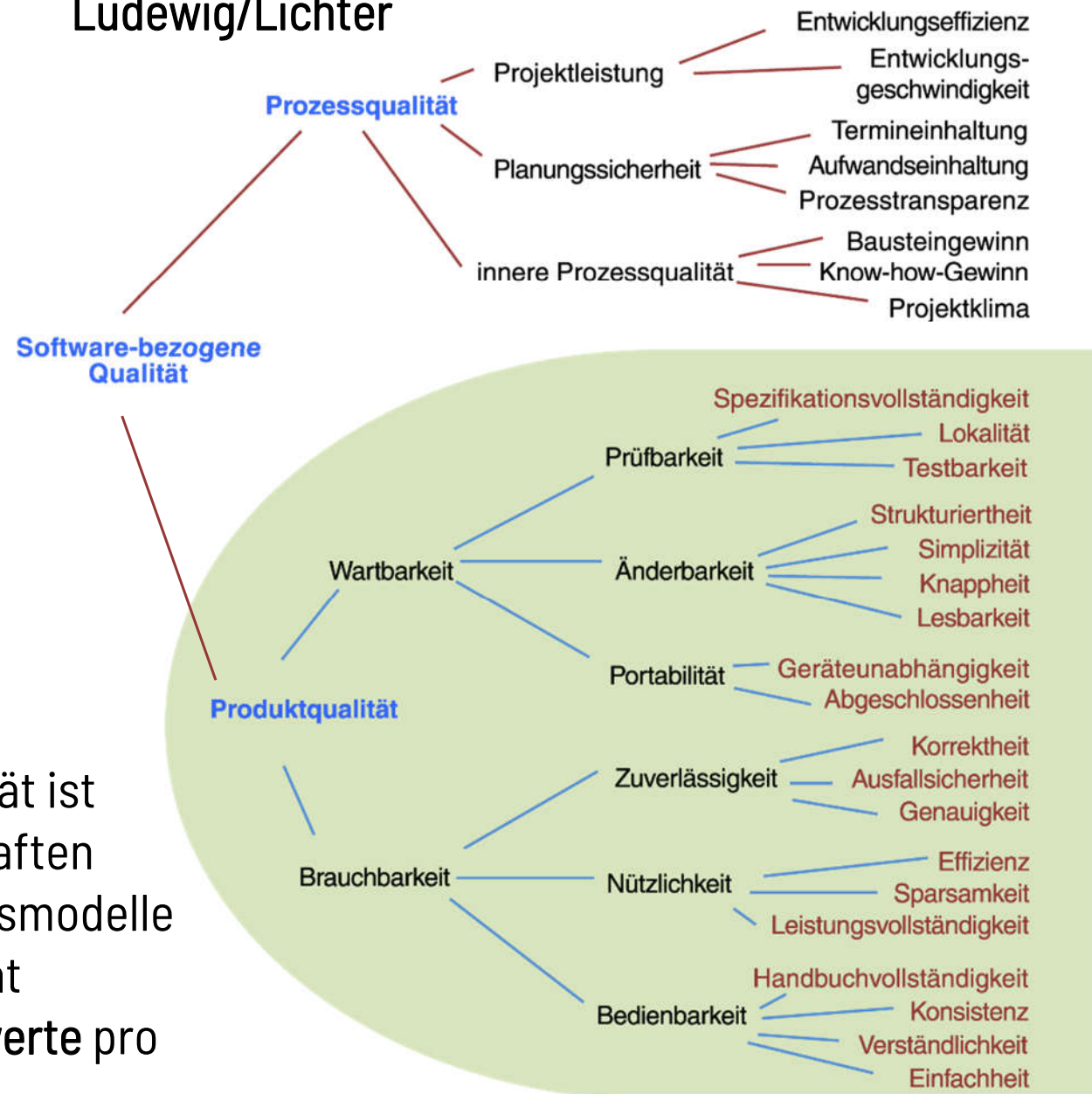
- Qualität allgemein oder Anforderungen, Bedürfnisse und Nutzungsbedingungen lassen sich in verschiedenen **Aspekte** betrachten, etwa: Funktionale Eignung, Wartbarkeit, Effizienz, Zuverlässigkeit, Benutzerfreundlichkeit, Sicherheit, ...
- Diesen Aspekten lassen sich **charakteristische Merkmale** zuordnen.
Beispiel Wartbarkeit: Wiederverwendbarkeit, Testbarkeit, Veränderbarkeit, Modularität
- Solche Merkmale sind eventuell wieder aufteilbar in Teilmerkmale
 - Das Ergebnis ist ein hierarchisches Modell (**Qualitätsmodell**), an dessen unterem Ende wir Merkmale (**Indikatoren**) erhalten, die wir messen können!
 - Damit können wir Qualität meß- und vergleichbar machen!

Qualitätsmodelle

Ein Qualitätsmodell zerlegt Qualität hierarchisch in einzelne Aspekte, diese in zugeordnete Merkmale und jeweils zugeordnete Teilmerkmale mit dem Ziel, messbare Größen (Indikatoren) zu erhalten.

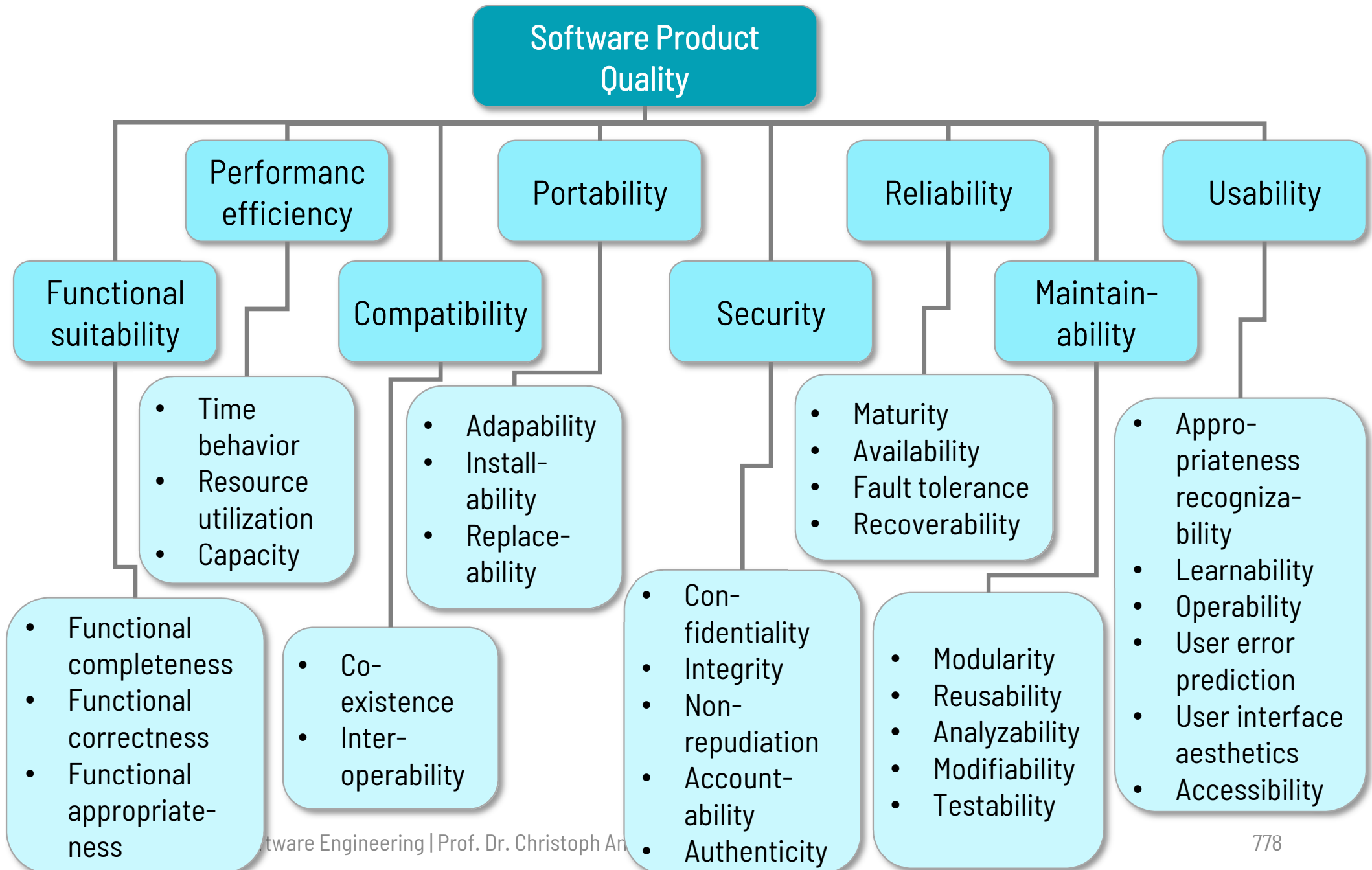
- Qualitätsmodelle zeigen: Qualität ist eine Sammlung ihrer Eigenschaften
- Qualität wird messbar: Qualitätsmodelle erlauben Steuerung von Qualität
- Dazu müssen Ziel- oder Grenzwerte pro Indikator festgelegt werden

Einfaches Qualitätsmodell für Software nach Ludewig/Lichter



Qualitätsmodelle

Qualitätsmodell nach ISO 25010



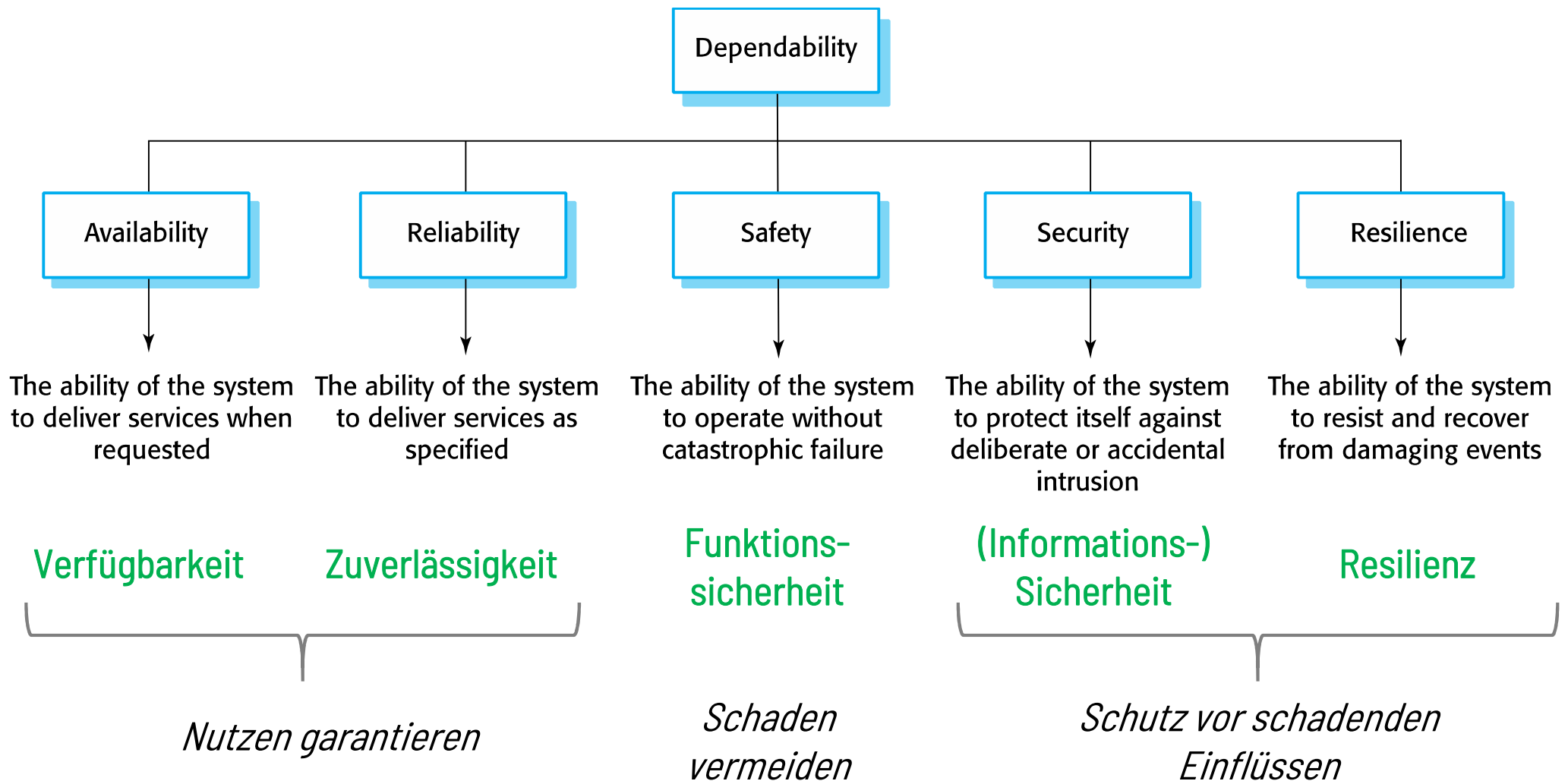
Verlässlichkeit („Dependability“) von Software

Ein zur Qualität verwandter Begriff

- Für viele Systeme eine sehr wichtige Eigenschaft!
- Spiegelt wieder, wie stark Benutzer einem System vertrauen und erwarten, dass sich das System bei normaler Nutzung wie erwartet verhält
- 1995 vorgeschlagen von Jean-Claude Laprie als Überbegriff für
 - Verfügbarkeit (*availability*)
 - Zuverlässigkeit (*reliability*)
 - Betriebssicherheit (*safety*)
 - (Informations-)Sicherheit (*security*)
- Diese Systemeigenschaften hängen voneinander ab

Verlässliche Systeme

Haupteigenschaften



SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 2)

Qualitätsprobleme

Wie entstehen Qualitätsprobleme?

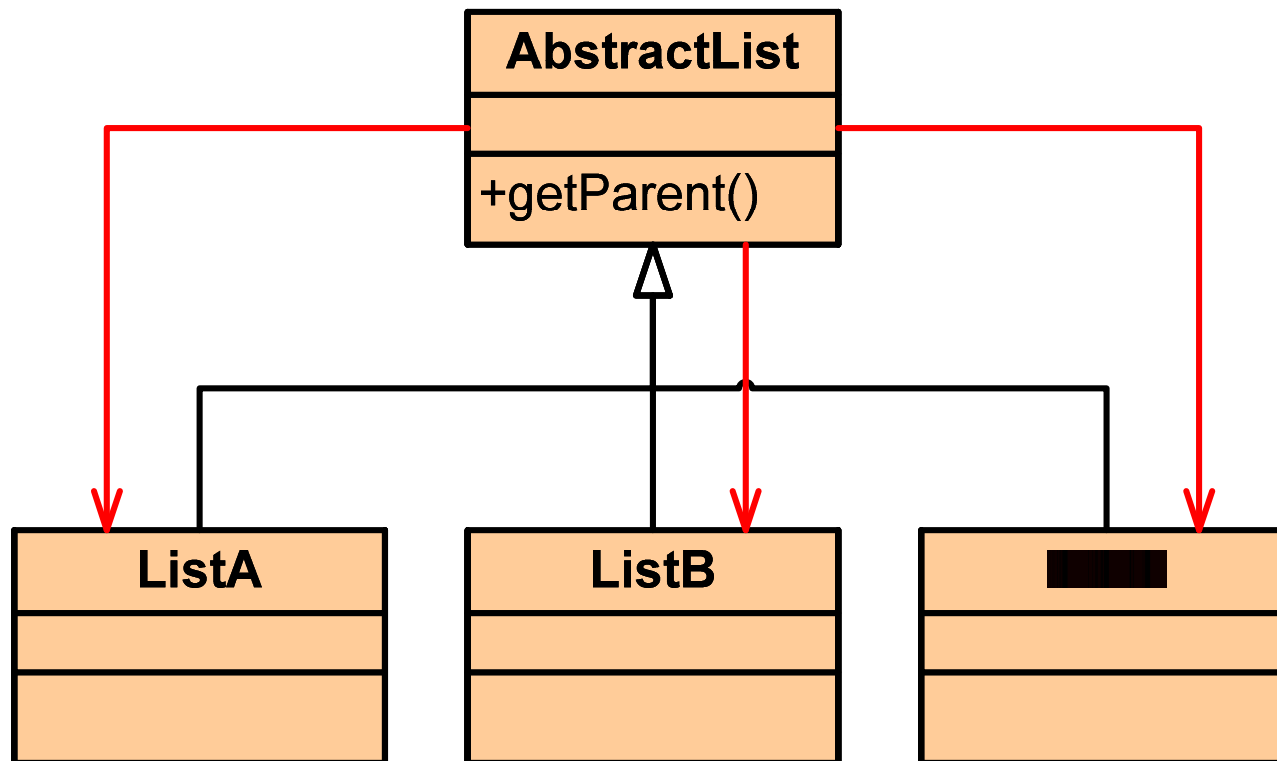
Vielfältige Ursachen sind möglich, darunter:

- **Schlechtes Anforderungsmanagement:** Unvollständig, Ungenau, Nicht abgestimmt
- **Schlechte Strukturen im Code:** Missachtung der SE-Prinzipien wie Abstraktion, Modularisierung (Kapselung, lose Koppelung, hohe Kohäsion, SoC), DRY
- **Allgemein: Unzureichendes Wissen** über Software-Entwicklung, die verwendeten Techniken und Werkzeuge oder über typische Fehler
- **Hohe Komplexität und ständige Veränderung:** in fachlichem Bereich, in technischem Bereich
- **Defizite im Projektmanagement:** Falsch verteilte Ressourcen
- **Mängel im Prozess- und Qualitätsmanagement**
 - Schlecht organisierte Prozesse
 - Unzureichende Qualitätsüberwachung



Beispiel: Qualitätsproblem bei Wartbarkeit

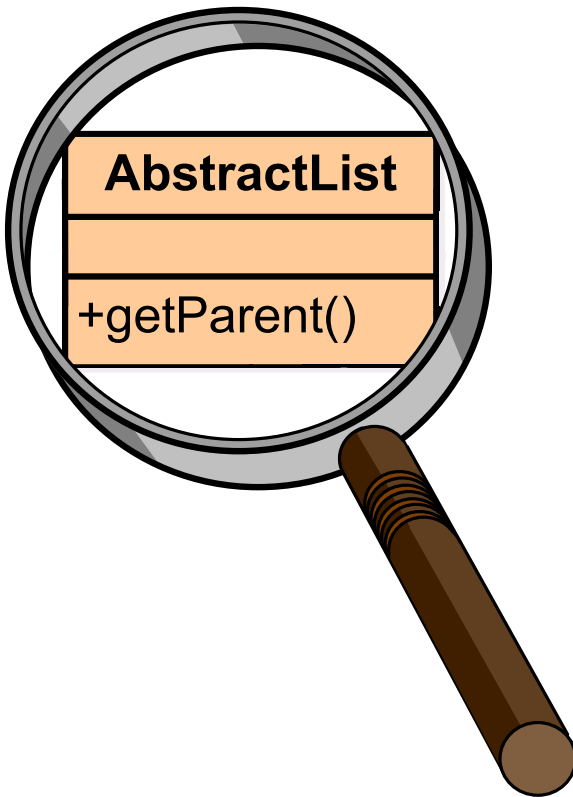
Eine Basisklasse kennt ihre Erben – schlecht weshalb?



Beispiel: Qualitätsproblem bei Wartbarkeit

Schlechte Strukturierung im Code

Maintain-
ability



```
public abstract class AbstractList {  
  
    // ...  
  
    public ListElement getParent(ListElement element) {  
        ListElement result = null;  
        if (this instanceof ListA) {  
            // ...  
            // find element's parent with ways of ListA  
            // ...  
        } else if (this instanceof ListB) {  
            // ...  
            // find element's parent with ways of ListB  
            // ...  
        } else if (this instanceof ListC) {  
            // ...  
            // find element's parent with ways of ListC  
            // ...  
        }  
        return result;  
    }  
  
    // ...  
}
```

Beispiel: Qualitätsproblem bei Wartbarkeit

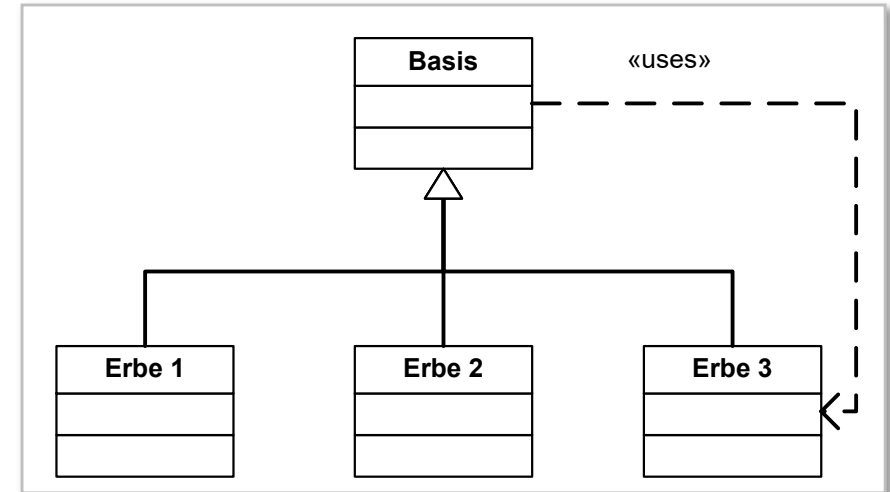
Schlechte Strukturierung im Code

Maintain-
ability



Eine Basisklasse,
die ihre Erben kennt

Verletztes Prinzip: Modularisierung.



- *Kapselung* von Basisklasse und abgeleiteten Klassen: Anstatt des Bezugs auf abgeleitete Klassen sollte polymorpher Aufruf verwendet werden
- *Koppelung*: Durch Verletzung der Kapselung werden die beteiligten Bauteile auch unnötig eng aneinander gekoppelt

 Veränderungen in den Erben zerbrechen wahrscheinlich die Basis.

Beispiel: Qualitätsproblem bei Wartbarkeit

Korrigierte Struktur



```
public abstract class AbstractList {  
    // ...  
  
    public abstract ListElement getParent(ListElement element);  
  
    // ...  
}
```

```
public class ListA extends AbstractList {  
  
    @Override  
    public ListElement getParent(ListElement element) {  
  
        // ...get the parent of the ListElement the  
        // way it is done in a ListA  
  
    }  
  
}
```

Beispiel: Qualitätsproblem bei Sicherheit SQL-Injection (1/5)

Security



Benutzername:

Passwort:

```
public class UserInterfaceHandler {  
    InputBox usernameBox;  
    InputBox passwordBox;  
    Database database;  
  
    public void submitButtonPressedForLogin() {  
        if (database.doLogin(usernameBox.getText(), passwordBox.getText())) {  
            enableAllFunctions();  
        } else {  
            showInvalidLoginErrorMsg();  
        }  
    }  
}
```


Beispiel: Qualitätsproblem bei Sicherheit SQL-Injection (2/5)

Security



```
public class UserInterfaceHandler {  
    InputBox usernameBox;  
    InputBox passwordBox;  
    Database database;  
  
    public void submitButtonPressedForLogin() {  
        if (database.doLogin(usernameBox.getText(), passwordBox.getText())) {  
            enableAllFunctions();  
        }  
    }  
}
```

```
public class Database {  
  
    public boolean doLogin(String user, String password) {  
        ResultSet rs;  
        try {  
            rs = executeQuery("SELECT * FROM user WHERE username = '"  
                             + user + "' AND password = '" + password + "'");  
  
            if (rs.next()) {  
                // Ergebnis enthält mindestens einen passenden Datensatz: Login erlaubt  
                return true;  
            } else {  
                // Benutzername nicht gefunden oder Passwort stimmt nicht, kein Login  
                return false;  
            }  
        } catch (SQLException e) {  
            // Datenbankfehler aufgetreten, Login nicht möglich  
            return false;  
        }  
    }  
}
```

Beispiel: Qualitätsproblem bei Sicherheit

SQL-Injection (3/5)

Security



Benutzername:

Passwort:

Anmelden

Beispiel: Qualitätsproblem bei Sicherheit SQL-Injection (4/5)

Security



```
public void submitButtonPressedForLogin() {  
    if (database.doLogin(usernameBox.getText(), passwordBox.getText())) {  
        enableAllFunctions();  
    } else {  
        // ...  
    }  
}
```

```
public boolean doLogin(String user, String password) {  
    ResultSet rs;  
    try {  
        rs = executeQuery("SELECT * FROM user WHERE username = '"  
                           + user + "' AND password = '" + password + "'");  
        if (rs.next()) {  
            // Ergebnis enthält mindestens einen passenden Datensatz: Login erlaubt  
            return true;  
        } else {  
            // Benutzername nicht gefunden oder Passwort stimmt nicht, kein Login  
            return false;  
        }  
    } catch (SQLException e) {  
        // ...  
    }  
}
```

```
rs = executeQuery("SELECT * FROM user WHERE username = '"  
                  + "' OR 1=1--" + "' AND password = '" + "" + "'");
```



```
rs = executeQuery("SELECT * FROM user WHERE username = '' OR 1=1--' AND password = ''");
```

- Benutzereingabe enthält SQL-Befehlsteile und wird ungeprüft weitergeleitet
 - In SQL: -- leitet Kommentar ein, Rest der Zeile wird ignoriert
- Login ohne Benutzername und ohne Passwort möglich!



Beispiel: Qualitätsproblem bei Sicherheit SQL-Injection (5/5)

- SQL-Injection, das Unterschieben von SQL-Befehlen in Eingabefelder oder Parameter, ist eine alte und immer noch häufige Schwachstellenart besonders bei Eingaben von der Benutzeroberfläche oder Zusammenarbeit zwischen Komponenten
 - 2021 wurden 591 SQL-Injection-Schwachstellen offiziell registriert (d.h. haben eine CVE-Nummer erhalten)
- Es gibt viele weitere typische Schwachstellen, darunter:
 - Cross-Site-Scripting (2021: 1.414 registrierte Fälle)
 - Buffer-Overflow (2021: 1.035 registrierte Fälle)

SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 3)

Sicherheit und Risiken

Sicherheit und Risiken

Was ist Sicherheit?

„Der Standard IEC 61508 *definiert Sicherheit als die Abwesenheit unakzeptabler Risiken.*

[...] *Hinreichende Abwesenheit von Gefährdungen durch Systeme* wird im englischen Sprachraum der Begriff **safety** verwendet.

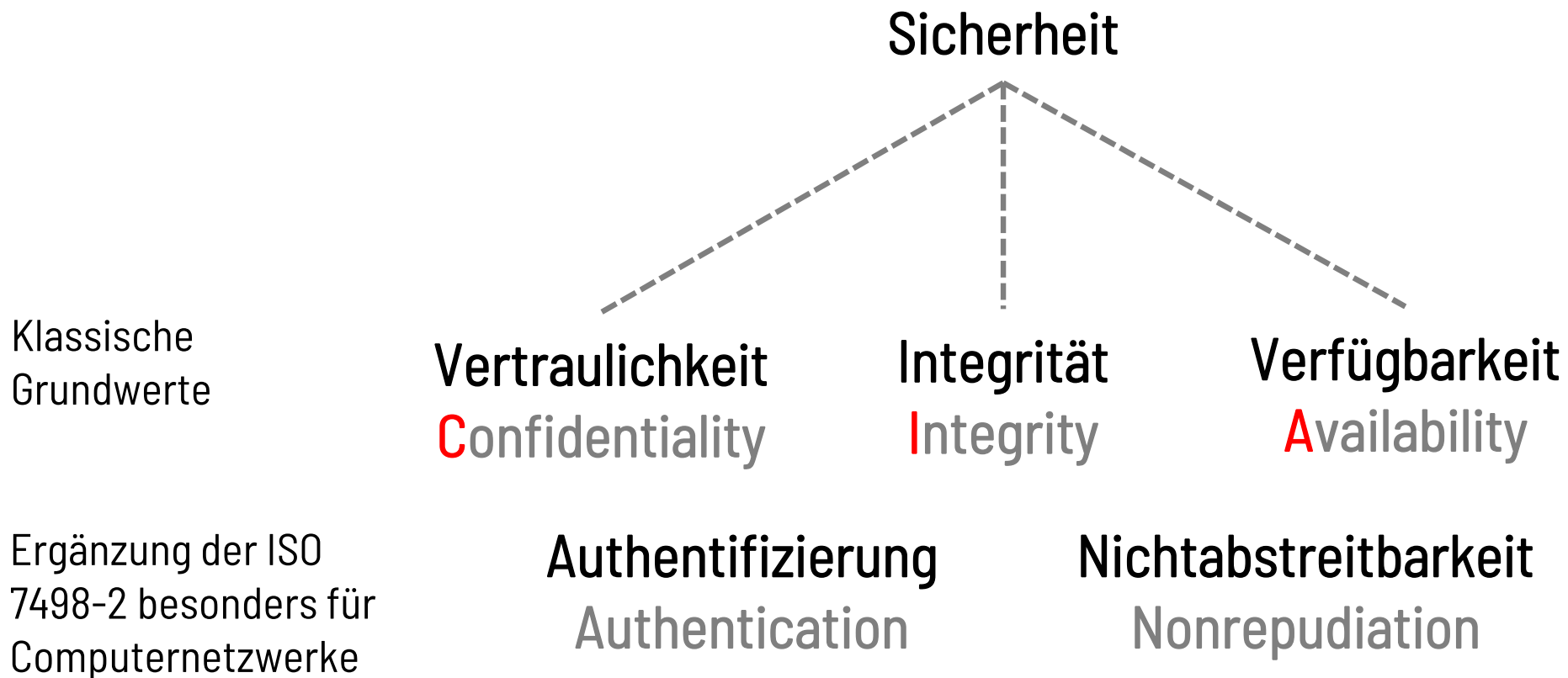
Darüber hinaus existiert die Eigenschaft **security**, die mit Datensicherheit übersetzt werden kann. Sie erfasst die *Eigenschaft eines Systems, Informationsverluste und unbefugten Datenzugriff zu verhindern.*

➤ **Sicherheit ist keine absolute Eigenschaft: Es gibt keine absolute Sicherheit. Sicherheit entsteht aus dem geeigneten Management von Risiken.**

Sicherheit und Risiken

Was ist Sicherheit?

Hier: Sicherheit im Sinne von „Informationssicherheit“ („Security“).



Sicherheit und Risiken

Welche Risiken? Einige Beispiele bei IT-Systemen

Katalog 47 elementarer Gefährdungen nach BSI IT-Grundschutz 200

- | | | |
|---|--|--|
| 1. Feuer | 17. Verlust von Geräten, Datenträgern oder Dokumenten | 30. Unberechtigte Nutzung oder Administration von Geräten und Systemen |
| 2. Ungünstige klimatische Bedingungen | 18. Fehlplanung oder fehlende Anpassung | 31. Fehlerhafte Nutzung oder Administration von Geräten und Systemen |
| 3. Wasser | 19. Offenlegung schützenswerter Informationen | 32. Missbrauch von Berechtigungen |
| 4. Verschmutzung, Staub, Korrosion | 20. Informationen oder Produkte aus unzuverlässiger Quelle | 33. Personalausfall |
| 5. Naturkatastrophen | 21. Manipulation von Hard- oder Software | 34. Anschlag |
| 6. Katastrophen im Umfeld | 22. Manipulation von Informationen | 35. Nötigung, Erpressung oder Korruption |
| 7. Großereignisse im Umfeld | 23. Unbefugtes Eindringen in IT-Systeme | 36. Identitätsdiebstahl |
| 8. Ausfall oder Störung der Stromversorgung | 24. Zerstörung von Geräten oder Datenträgern | 37. Abstreiten von Handlungen |
| 9. Ausfall oder Störung von Kommunikations-netzen | 25. Ausfall von Geräten oder Systemen | 38. Missbrauch personenbezogener Daten |
| 10. Ausfall oder Störung von Versorgungsnetzen | 26. Fehlfunktion von Geräten oder Systemen | 39. Schadprogramme |
| 11. Ausfall oder Störung von Dienstleistern | 27. Ressourcenmangel | 40. Verhinderung von Diensten (Denial of Service) |
| 12. Elektromagnetische Störstrahlung | 28. Software-Schwachstellen oder -Fehler | 41. Sabotage |
| 13. Abfangen kompromittierender Strahlung | 29. Verstoß gegen Gesetze oder Regelungen | 42. Social Engineering |
| 14. Ausspähen von Informationen (Spionage) | | 43. Einspielen von Nachrichten |
| 15. Abhören | | 44. Unbefugtes Eindringen in Räumlichkeiten |
| 16. Diebstahl von Geräten, Datenträgern oder Dokumenten | | 45. Datenverlust |
| | | 46. Integritätsverlust schützenswerter Informationen |
| | | 47. Schädliche Seiteneffekte IT-gestützter Angriffe |

Sicherheit und Risiken

Welche Risiken? Einige Beispiele bei Web-Anwendungen

OWASP Top Ten 2021:

- A01 Broken Access Control
- A02 Cryptographic Failures
- A03 Injection
- A04 Insecure Design
- A05 Security Misconfiguration
- A06 Vulnerable and Outdated Components
- A07 Identification and Authentication Failures
- A08 Software and Data Integrity Failures
- A09 Security Logging and Monitoring Failures
- A10 Server Side Request Forgery

Konstruktive Sicherheit

Welche Risiken? Technische Ebenen

Application

Reusable components and libraries

Middleware

Database management

Generic, shared applications (browsers, e--mail, etc)

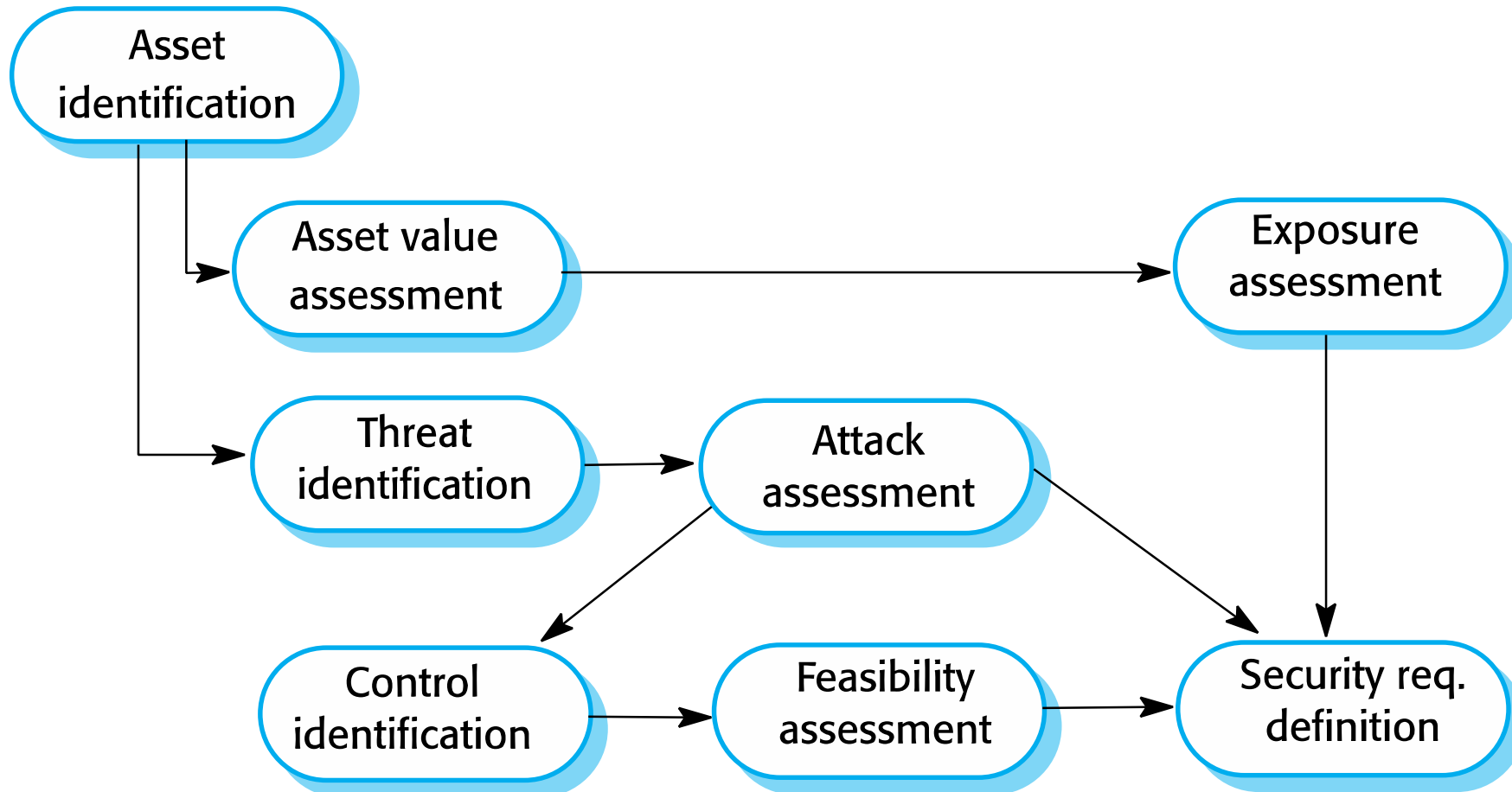
Operating System

Network

Computer hardware

Konstruktive Sicherheit

Risikoanalyse als Teil des Entwurfs



- Das Ergebnis einer solchen Risikoanalyse sind **Sicherheitsanforderungen**
- Erste Sicherheitsanforderungen sollten bereits Teil der Anforderungsaufnahme sein

Konstruktive Sicherheit

IT-Sicherheit: Ein paar Grundbegriffe

Deutsch	Englisch	Definition
(Vermögenswert)	Asset	Etwas wertvolles, das geschützt werden muss: Software, Daten, Systeme u.a.
Angriff	Attack	Ausnutzung der Schwachstelle eines Systems, im allgemeinen von außerhalb des Systems und mit Schadensabsicht
(Regelung)	Control	Schutzmaßnahme um eine Schwachstelle zu mindern, etwa Verschlüsselung
Gefahrenpotenzial	Exposure	Mögliche Verluste (Datenverlust, Zeitverlust, nötiger Aufwand bei Wiederherstellung nach einem Sicherheitsvorfall) oder Schaden bei einem Rechnersystem.
Bedrohung	Threat	Konkrete Umstände mit dem Potential haben, Verluste oder Schaden zu produzieren: Etwa Systemschwachstelle, die einem Angriff ausgesetzt ist.
	Advanced Persistent Threat (APT)	Gezielter Angriff mit komplexen Angriffstechnologien- und Taktiken sowie aufwendigen Hintergrundinformationen eines Opfer-IT-Systems und dessen Umgebung. Der Angreifer nimmt großen Aufwand auf sich (Advanced), um erfolgreich auf ein Opfer-IT-System zuzugreifen und möglichst lange (Persistent) unentdeckt zu bleiben.
Schwachstelle	Vulnerability	Eine Schwachstelle in einem System, die ausgenutzt werden kann, um Verluste oder Schaden zu verursachen.

SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 4)

Qualitätssicherung für Software

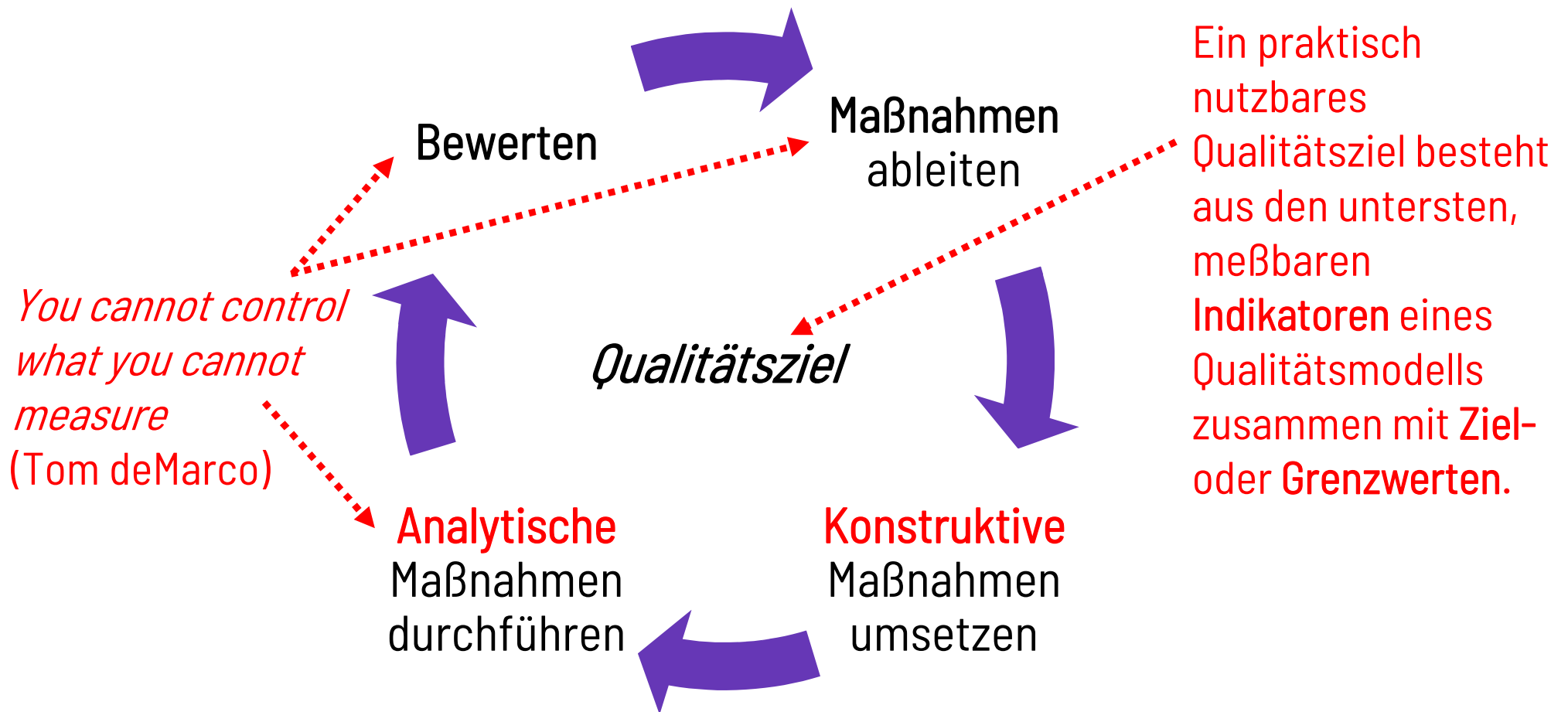
Softwarequalitätsmanagement

Managementmaßnahmen zur Sicherstellung des notwendigen Qualitätsniveaus für ein Softwareprodukt.

- Auf **Organisationsebene** soll der Rahmen für organisatorische Prozesse und Standards geschaffen werden, der zu hoher Qualität führt
- Auf **Projektebene**:
 - Aufstellung eines Qualitätsplans, der Qualitätsziele, zugehörige Prozesse und Standards für das Projekt festlegt
 - Anwendung spezifischer Qualitätsprozesse und deren Überwachung
- Die **Norm EN ISO 9001** legt Anforderungen an Qualitätsmanagementsysteme allgemein fest

Qualitätssicherung für Software

Eine einfache Vorgehensweise



Qualitätssicherung für Software

konstruktive Maßnahmen

Konstruktive Maßnahmen:

- Methoden (Prinzipien des Software Engineerings)
- Prozesse
- Werkzeuge
- Richtlinien, Standards, Checklisten

Beispiele:

- Prinzipien zu Abstraktion, Modularisierung, DRY
- typisierte Programmiersprachen
- Programmier- und Namenskonventionen
- Automatisierung fehlerträchtiger manueller Schritte
- Beständige Prozessverbesserung
- ...

Praktisch der
komplette bisherige
Inhalt der
Lehrveranstaltung,
plus einiges mehr

Qualitätssicherung für Software

analytische Maßnahmen

Analytische Maßnahmen:

- Ohne Ausführung der Software: Analyse auf Basis von Quelltext und Modellen
 - Softwaremaße („Metriken“)
 - Prüfung von Heuristiken
- Mit Ausführung der Software:
 - Testen
 - Profiling
 - Schwachstellenanalyse

zielt *ehrer* Richtung Qualität
aus Sicht des Entwicklers

zielt *ehrer* Richtung Qualität
aus Sicht des Nutzers

Analytische Maßnahmen

Profiling von Software

Beobachtete Ausführung mit Aufzeichnung des Laufzeitverhaltens von Software zur Analyse von

- Ausführungszeiten
- Aufrufhäufigkeiten
- Speichernutzung
- Nebenläufigkeit

Werkzeugbeispiele für Java:

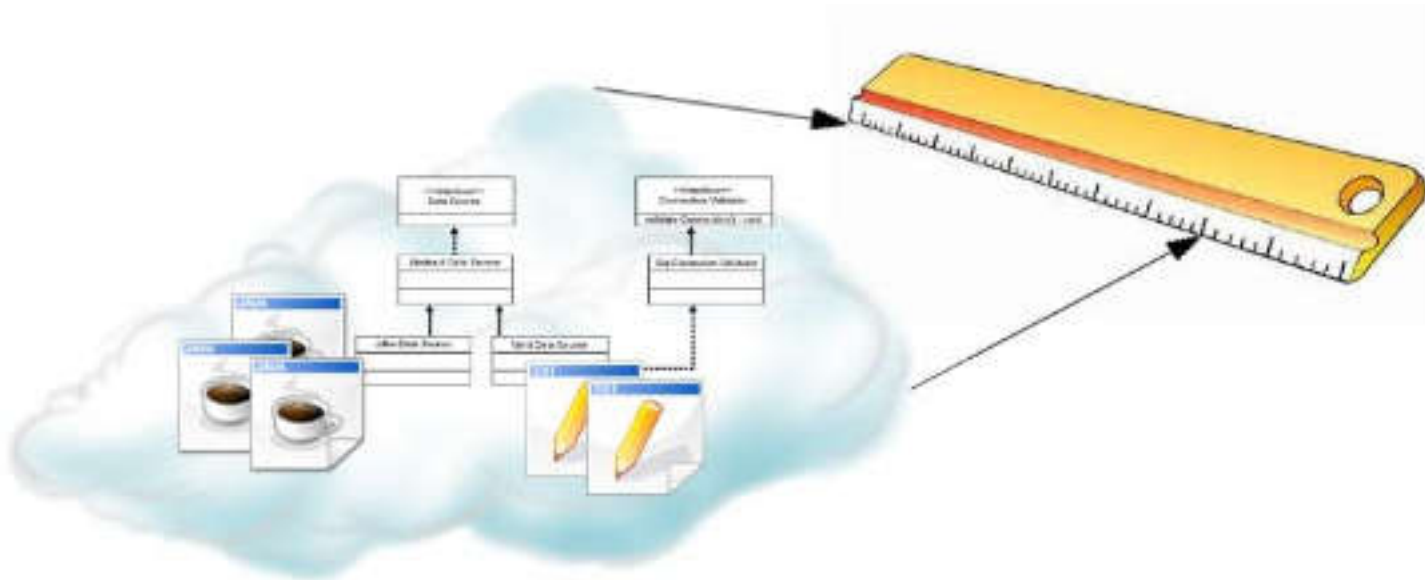
- Java Flight Recorder
- Visual VM

Analytische Maßnahmen

Softwaremaße („Metriken“)

Maße zur Quantifizierung von Softwareelementen

- **Ziel:** Beurteilung von Elementen zur Schwachstellensuche
- **Wie:** Zuordnung von Elementen zu Werten auf einer Skala



Softwaremaße

Lines of Code

Bestimmen Sie die Anzahl der Quelltextzeilen!

```
1 package de.htwg.swen.metriken;
2
3 /**
4  *
5  * Eine Beispielsklasse
6  */
7 public class LoC {
8     // Ein Kommentar
9 }
10
```

1

```
1 package de.htwg.swen.metriken;
2
3 /**
4  *
5  * Eine Beispielsklasse
6  */
7 public class LoC {
8     // Ein Kommentar
9     private String name;
10
11     public LoC() {
12         name = "";
13     }
14 }
```

2

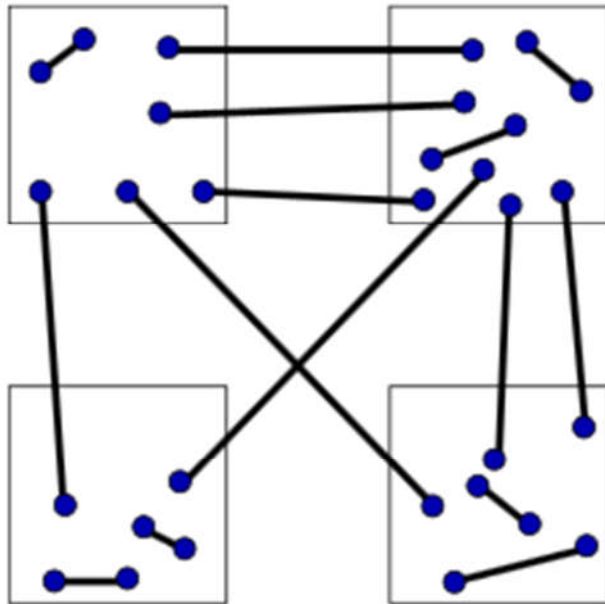
Softwaremaße

Lines of Code

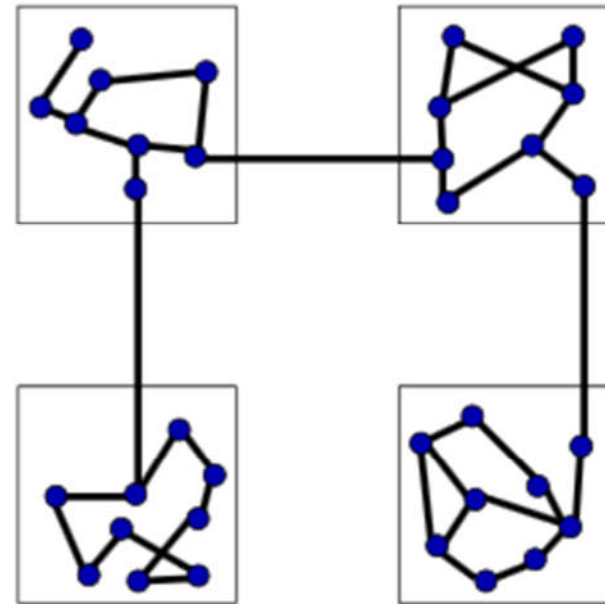
- Ziel: Zeilen des Quellcodes zählen
- Diskussion:
 - Einfach zu ermitteln
 - Bestimmt einen Umfang
 - Ist sprachabhängig
 - Ist programmierstilabhängig
 - Keine Aussage über Komplexität
 - Unterschiedliche Interpretationen
 - Alle | ausführbare Programmzeilen
 - Mit oder ohne Leerzeilen und Kommentare

Kopplung und Kohäsion

Wie kann man diese Maße messen?



starke Kopplung
schwache Kohäsion



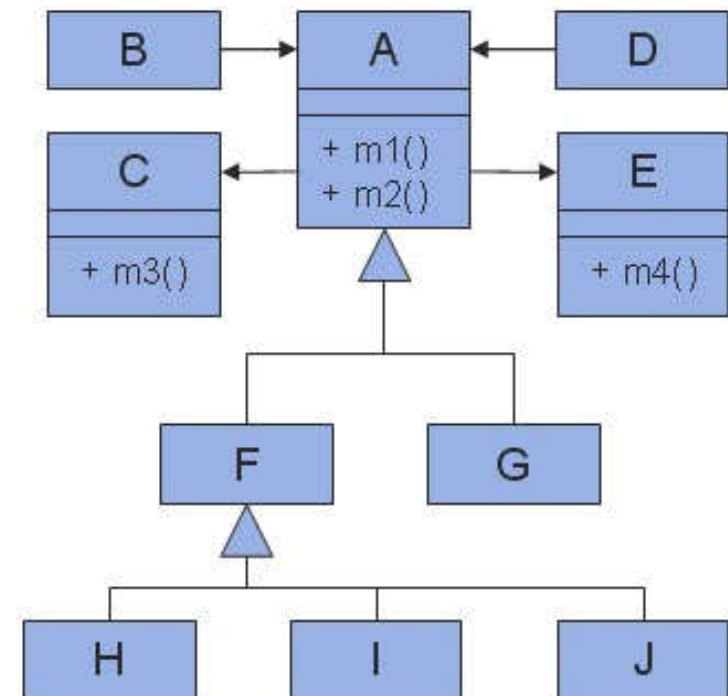
schwache Kopplung
starke Kohäsion



Softwaremaße

Kopplung zwischen Klassen (CBO)

- Nach Chidamber / Kemerer
- Zählt in einer Klasse Aufrufe anderer Klassen (über Methodenaufrufe, Attribute, Methodenparameter)
- Zeigt damit Anzahl der Klassen, mit denen kommuniziert wird
- Indikator für
 - starke **Kopplung**
 - großen Verwaltungs- und Testaufwand
 - schlechte „Aufgabenverteilung“

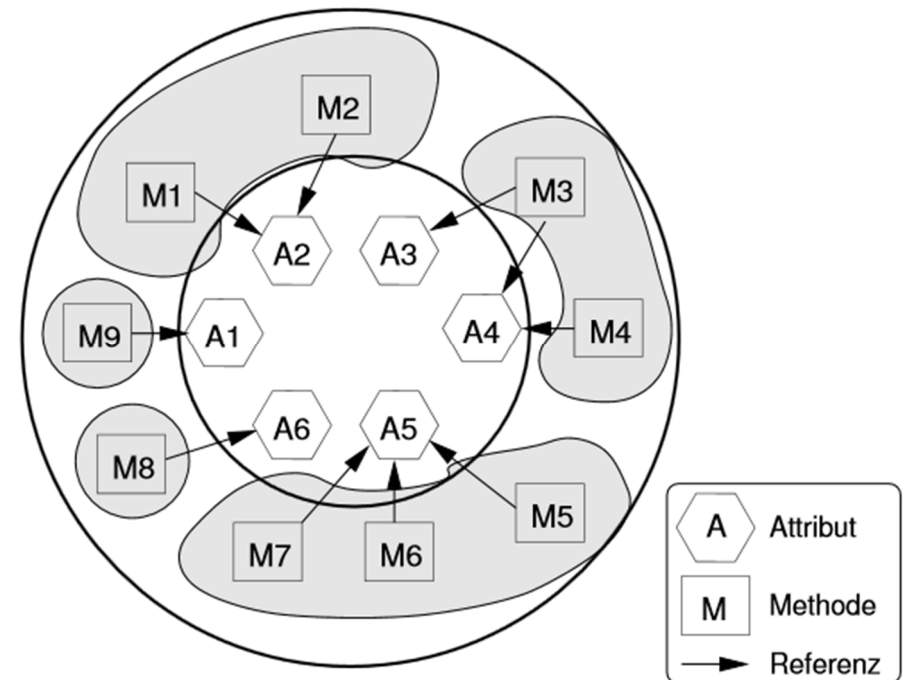


Softwaremaße

Lack of cohesion on methods (LCOM1)

Anhang

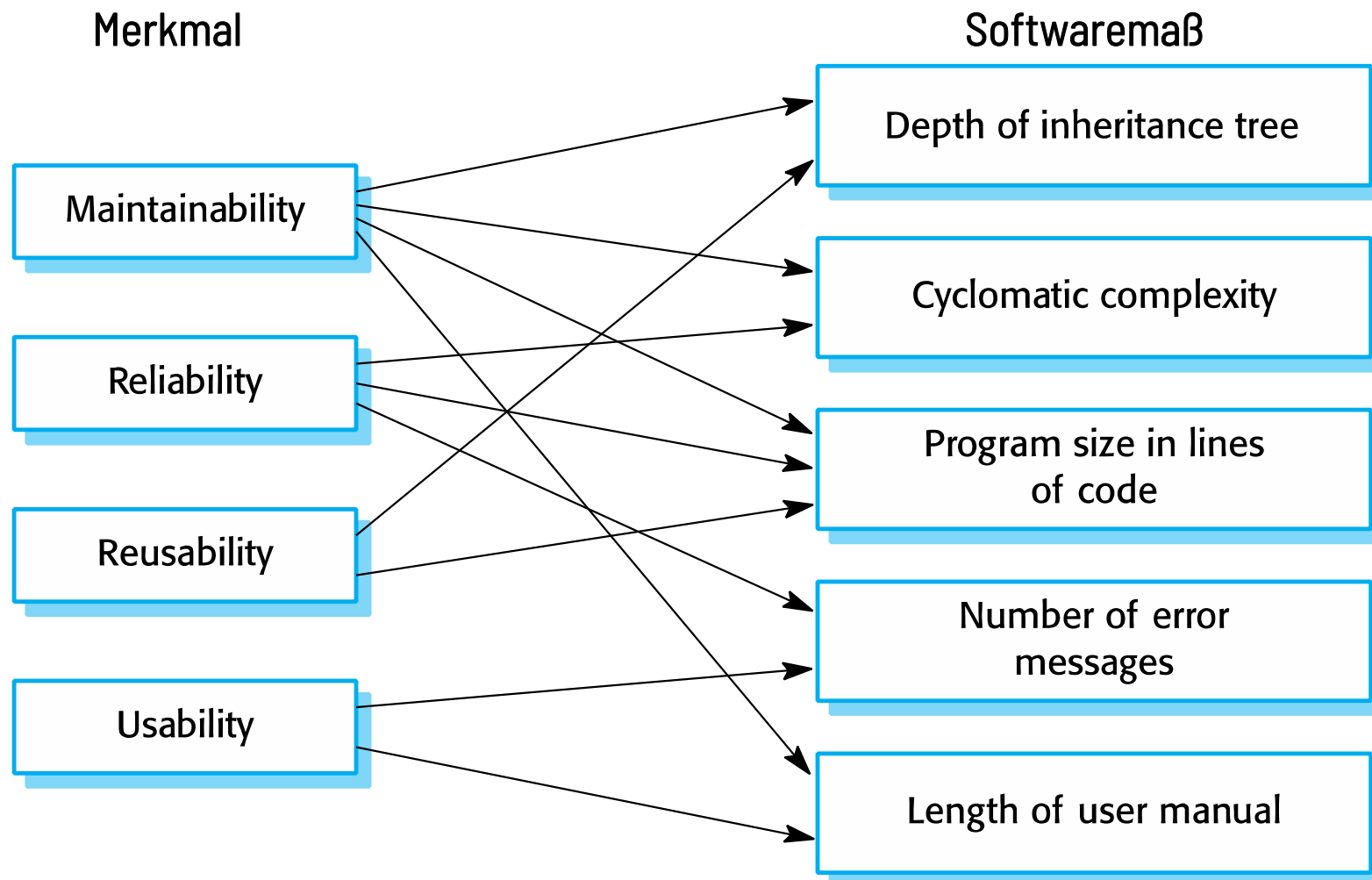
- Nach Chidamber / Kemerer: Messung von Kohäsionsmängeln der Methoden einer Klasse
- $LCOM1 = \max(P - Q, 0)$
 - P = Anzahl der Paare von Methoden einer Klasse, die disjunkte Instanzvariablen benutzen;
 - Q = Anzahl ... die mind. 1 Variable gemeinsam benutzen
- Indikator für
 - schlechtes Design
 - schlechte Klassenaufteilung
 - mögliches Trennen in mehrere Klassen



- 36 Methoden-Paare (8 mit M9 + 7 mit M8 + 6 mit M7 + ... + 1 mit M2)
- Q : 5 Paare (M7/M6, M7/M5, M6/M5, M3/M4, M1/M2)
- P : Alle anderen $\rightarrow 31$
- $LCOM1 = 31 - 5 = 26$

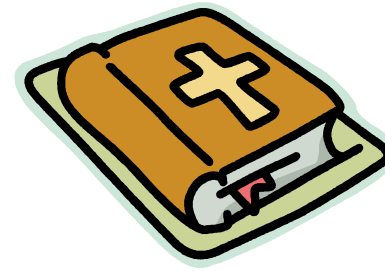
Qualitätsmodelle und Softwaremaße

Softwaremaße messen Qualitätsmerkmale



Analytische Verfahren

Heuristiken



(Unbewiesene) Regeln oder Hypothesen zum Vorgehen oder zur Erkenntnisgewinnung, als Gebote oder Verbote

Im Software-Entwurf:

Heuristiken sind Konkretisierungen von (Entwurfs- oder Software Engineering-)prinzipien in speziellen Kontexten

- **Beispiel** – Lose Kopplung bei OO-Vererbung: Klassen sollten nicht von Unterklassen abhängen
- Heuristiken sind **formalisierbar** und **automatisch prüfbar**: Oft über Einsatz von Softwaremaßen mit Schwellwerten

Beispiele für Heuristiken

Diese Heuristiken sind noch nicht formalisiert, es fehlen überwiegend Schwellwerte.

Anhang Heuristiken des objektorientierten Entwurfs Entwurf von Klassen und Objekten

- Eine Klasse sollte genau eine Abstraktion (Verantwortlichkeit) realisieren

Anhang Heuristiken des objektorientierten Entwurfs Entwurf von Klassen und Objekten

- Benutzen Sie Attribute, um Veränderungen von Wert auszudrücken. Um Veränderung im Verhalten auszudrücken können Sie das Überschreiben von Methoden verwenden.
- Vermeiden Sie es den Typ eines Objekts zur Laufzeit zu ändern.
- Halten Sie sich bei der Modellierung von Klassen möglichst an der realen Welt. Streben Sie bei Analyse und Design nach strukturellen Ähnlichkeiten.
- Vermeiden Sie überlange Argumentlisten. Darunter leidet die Übersichtlichkeit von Methodenaufrufen.

Anhang Heuristiken des objektorientierten Entwurfs Beziehungen zwischen Klassen und Objekten

- Erben Sie nur von abstrakten Klassen oder Interfaces:
Überschreiben von Funktionalität kann kontraproduktiv sein.

Anhang Heuristiken des objektorientierten Entwurfs Beziehungen zwischen Klassen und Objekten

- Minimieren sie die Abhängigkeiten einer Klasse.
- „Reden Sie nicht mit Fremden“, d.h. ein Objekt sollte nur sich selbst, seine Attribute oder die Argumente seiner Methoden referenzieren

Die Klasse DemeterSzenario kennt (nutzt) den Referee gar nicht und sollte ihn deshalb nicht referenzieren.

```
DemeterSzenario.java | FootballGame.java | Referee.java | Team.java | 20
1 package demeter;
2
3 /**
4  *
5  * Law of Demeter
6  */
7 public class DemeterSzenario {
8
9     public static void main(String[] args) {
10
11         Team gewinner = new Team("FC Schalke");
12         Team verlierer = new Team("FC Bayern");
13
14         FootballGame game = new FootballGame(gewinner, verlierer);
15
16         // Die Segel hochheben
17         game.getReferee().startGame();
18     }
19 }
```

Analytische Verfahren

Weitere Beispiel für Werkzeuge

Es sind zahlreiche Werkzeugen zu unterschiedlichen Aspekten analytischer Methoden verfügbar.

Beispiele:

- Eclipse Metrics (Eclipse-Plugin)
- Checkstyle (Plugin für Eclipse, Maven)
- Sonarqube
- Infusion
- Burpsuite

• ...



Sonarqube

<https://sonarcloud.io/>

sonarcloud

My Projects

My Issues

+

Q

XP

XWiki Platform - Parent POM

☆

Overview

Main Branch

Pull Requests0

Branches3

Information

Collapse

XWiki / XWiki Platform - Parent POM / master

SummaryIssuesSecurity HotspotsMeasuresCodeActivity

298k Lines of CodeVersion 14.3-SNAPSHOTLast analysis 3 hours ago: dfb254772 warnings

Quality Gate ?
Passed

New Code

Overall Code

RELIABILITY

249 Bugs

E

MAINTAINABILITY

12k Code Smells

A

SECURITY

168 Vulnerabilities

E

SECURITY REVIEW

140 Security Hotspots

0.0% Reviewed

E

COVERAGE

45.8% Coverage

DUPLICATIONS

0.9% Duplications

© 2008-2022, SonarCloud by SonarSource SA. All rights reserved.

News - Twitter - Terms - Pricing - Privacy - Security - Help - Contact us - Status - About

sonarcloud

My Projects

My Issues

+

Q

XP

XWiki Platform - Parent POM

☆

Overview

Main Branch

Pull Requests 0

Branches 3

Information

XWiki / XWiki Platform - Parent POM / master

Summary

Issues

Security Hotspots

Measures

Code

Activity

Passed

Project Overview

Releasability

Reliability

Security

Security Review

Maintainability

Overview

On new code

Code Smells 106

Debt 2d

Debt Ratio 0.4%

Rating A

Overall

Code Smells 11,905

Debt 223d

Debt Ratio 1.2%

Rating A

Effort to Reach A 0

Coverage

Duplications

Size

Complexity

Issues

XWiki Platform - Parent POM

500 / 5,635 files

New code: last 30 days

Risk

Color: Worse of Reliability Rating and Security Rating Size: Lines of Code

A B C D E

Coverage

0.0%

20.0%

40.0%

60.0%

80.0%

100%

0

2d

4d 1h

6d 2h

8d 2h

Technical Debt

← Collapse

V23.1

Software Engineering | Prof. Dr. Christoph Andriessens

© 2008-2022, SonarCloud by SonarSource SA. All rights reserved.

817

Zusammenfassung (1/2)

- Fehler und Schwachstellen in Software sind ein großes Problem bei der Bedeutung, die Software in unserer heutigen vernetzten Welt hat
- Softwarequalität wird durch Qualitätsmodelle und darauf aufbauende Qualitätsziele mess- und steuerbar gemacht
- Sicherheit als ein Qualitätsmerkmal ist nie absolut, entsteht durch hinreichend gutes Management von Risiken



Zusammenfassung (2/2)

- Software Engineering beinhaltet viele Prinzipien, Methoden und Werkzeuge zur konstruktiven Qualitätssicherung
- Zu besonders wichtigen analytischen Werkzeugen gehören Tests, Softwaremaße und Heuristiken
- Werkzeuge unterstützen bei analytischen Methoden und verschaffen wichtige Informationen über die Qualität von Implementierung und Entwurf, sind aber in der Bewertung kritisch zu hinterfragen



Software durchdringt die vernetzte Welt

Quellen

- <https://www.heise.de/news/Softwarefehler-Radiosender-in-Seattle-schrottet-Infotainment-von-Mazda-6446140.html>
- <https://www.heise.de/news/Y2K22-Bug-legt-Stromanschluss-lahm-Kaerntner-ohne-Warmwasser-6317000.html>
- <https://www.heise.de/news/Ransomware-Erpressung-mehr-als-verdoppelt-2021-ueber-600-Millionen-Angriffe-6524397.html>
- <https://www.heise.de/news/Cyberattacken-auf-Ukraine-6524405.html>